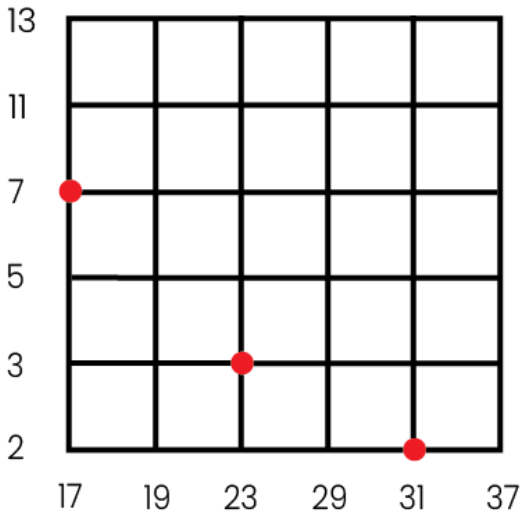
# Ritangle 2026

## Stage 1 - Question 10

Project $\pi$*

October 17, 2026

Topic - Combinatorics




*Solved by Nihal Sharma.

# 1 Problem

The problem statement was as follows:

*The three points in the 5 by 5 square grid below are positioned (where grid lines meet) so that the distance between any pair of them is unique. These distances are* $\sqrt{5}$ , $\sqrt{8}$ *and* 5 *units. The value assigned to each point is the product of the primes at the end of its row and column; the values of the points are 62, 69 and 119.*

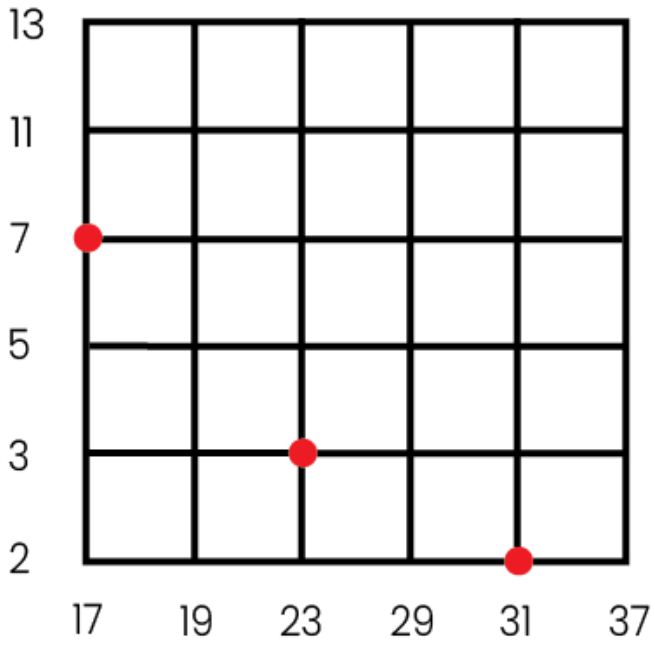



Figure 1: Reference image

*Add three more points so that each distance between any two of the six points is unique; in other words, the six points determine 15 different distances.*

*What is the sum of the values of the three new points?*

# 2 Solution

Let us define the grid as a co-ordinate axis, with the bottom left point being $(0, 0)$.

From this we have defined 3 points from the problem statement, $P_1 = (0, 3), P_2 = (2, 1), P_3 = (4, 0)$

To find potential candidate grid points, we must eliminate non-candidates which can be done via removing all points which are any distance $d$ away from $P_i$, where $d \in \{\sqrt{5}, \sqrt{8}, 5\}$, $i \in \{1, 2, 3\}$ as these will result in at least one repeated distance, leading to less than 15 unique distances

We can do this with a simple equation:

$$(x - x_{P_i})^2 + (y - y_{P_i})^2 = d^2 \text{ where } x \text{ and } y \in \mathbb{N}$$

From this we attain the following candidate points,

$$(0, 1), (0, 4), (0, 5), (1, 0), (1, 2), (2, 0),$$
$$(2, 3), (3, 0), (3, 1), (3, 4), (3, 5), (4, 1),$$
$$(4, 4), (5, 0), (5, 1), (5, 4)$$

From this we can apply the following algorithm:

1. For every triple (set of 3) candidate points from the above list, together with the 3 defined points this will be the 6 points, which we will denote as $P$
2. Let $D$ represent the set of distances between the points, for now $D$ is empty
3. For each pair of points in $P$, calculate their distance
   (a) If this distance is in $D$, then we have a repeated distance, hence the 3 candidate points chosen cannot form a valid group of 6
   (b) If it is not in $D$, then place it in $D$
4. If all pairs of $P$ had no repeating distances, then we have found a valid triple

Performing this algorithm yields $P = \{(0, 5), (3, 5), (5, 0)\}$

Recalling from Figure 1 and the rules of the problem statement, $(0, 5)$ has a value of $13 \times 17 = 221$, $(3, 5)$ has a value of $29 \times 13 = 377$, and $(5, 0)$ has a value of $37 \times 2 = 74$

Summing these values gives an answer of **672**.

**Final Answer Summary:**
Sum of Values = 672 $\implies$ *Hint: A=1*

# 3 Appendix

```python
import itertools


PRIME_X = (17, 19, 23, 29, 31, 37)
PRIME_Y = (2, 3, 5, 7, 11, 13)


class Point:
    def __init__(self, x: int, y: int) -> None:
        self.x = x
        self.y = y
        self.prime_x = PRIME_X[x]
        self.prime_y = PRIME_Y[y]

    def __str__(self) -> str:
        return f"({self.x}, {self.y})"

    def value(self) -> int:
        return self.prime_x * self.prime_y


def distance_squared(point_a: Point, point_b: Point) -> int:
    return (point_a.x - point_b.x) ** 2 + (point_a.y - point_b.y) ** 2


POINTS = (Point(0, 3), Point(2, 1), Point(4, 0))
DISTANCES_SQUARED = (5, 8, 25)


def unique_distance_check(point: Point) -> bool:
    return all(
        distance_squared(point, known_point) not in DISTANCES_SQUARED
        for known_point in POINTS
    )


def filter_grid() -> list[Point]:
    candidate_points = []
    for x in range(6):
        for y in range(6):
            candidate = Point(x, y)
            if unique_distance_check(candidate):
                candidate_points.append(candidate)
    return candidate_points


def brute_force() -> None:
    candidates = filter_grid()
```

```
    for combination in itertools.combinations(candidates, 3):
        all_points = [*combination, *POINTS]
        distances = {
            distance_squared(point_a, point_b)
            for point_a, point_b in itertools.combinations(all_points, 2)
        }

        if len(distances) == 15:
            point_a, point_b, point_c = combination
            print()
            print(f"{point_a}, {point_b}, {point_c} is a valid triple")
            print(f"Summed value: {point_a.value() + point_b.value() +
               point_c.value()}")


brute_force()
```